Operating Systems
Chapter 5

Concurrency: Mutual Exclusion and Synchronization

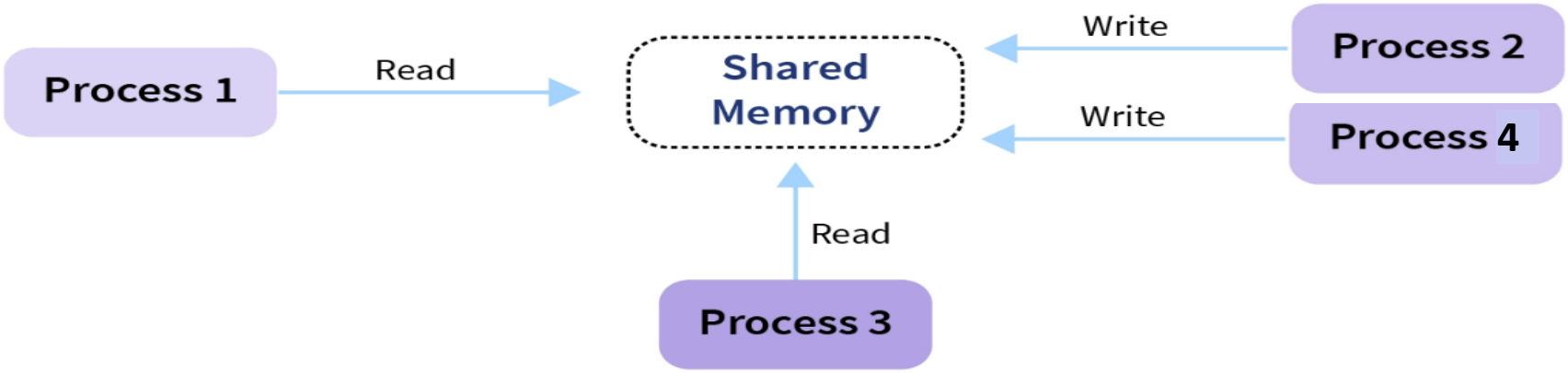
Instructor

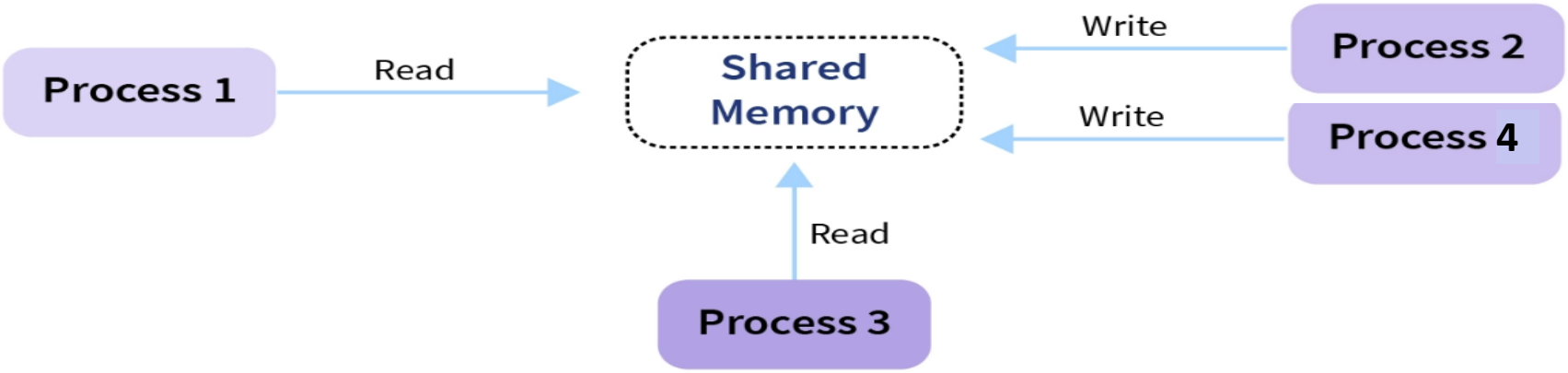
Dr.Eng. Mahmoud M. Ismail

Outline

- Principles of Concurrency
- Mutual Exclusion Software Approaches
- Mutual Exclusion: Hardware Support
- Mutual Exclusion: OS and programming language support
  - Semaphores

- In a uniprocessor multiprogramming system, processes are **interleaved** in time to yield **the appearance of simultaneous execution**.
- In a multiple-processor system, it is possible not only to **interleave processes** but also to **overlap** them
- Process or thread may **run simultaneously** with other processes or threads, which thus **raises an issue of concurrency**.
- **Concurrency** means **the concurrent execution** of processes
  - All processes **Read**
  - One process **write** and others **Read** → **order**
- **Concurrent access** to **shared data** may result in **data inconsistency** (**Read** and **write** at the same time or **multiples writes** at the same time)





- Maintaining **data consistency** requires mechanisms to ensure organized execution of **cooperating processes**.
- The basic requirement for support of **concurrent execution** of processes is the ability to enforce **mutual exclusion**.

Concurrency

Concurrency affects a number of design issues

- Communication among processes
- Sharing resources
- Synchronization (the operation or activity of two or more things at the same time) of the activities of multiple processes
- Allocation of processor time

Concurrency

Concurrency arises in three different contexts:

- Multiple applications
  - **Multiprogramming** allows **processing time** to be dynamically shared among a number of active applications
- Structured application
  - **Application** can be a set of **concurrent processes**
- Operating-system structure
  - **Operating system** is **a set of processes or threads**

Difficulties with Concurrency

The relative speed of execution of *processes* *cannot be predicted*, it depends on

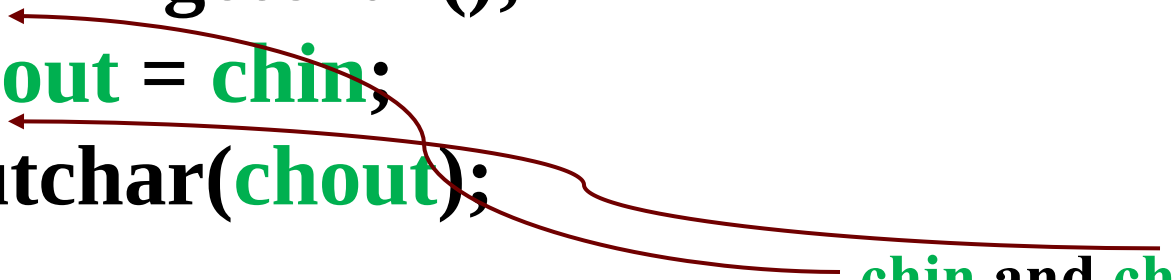
1) The activities of other processes
2) The way in which the OS handles interrupts,
3) The scheduling policies of the OS.

The following difficulties arise:

1) Sharing global resources
   - Two processes access the same global variable
   - ➢ The order in which the various **reads** and **writes** are executed is critical.
2) Management of allocation of resources
   - A process is allocated an I/O channel and then is suspended before using that channel.
   - ➢ It may be undesirable for OS simply to lock the channel and prevent its use by other processes; indeed, this may lead to a deadlock condition.
3) Programming errors difficult to locate

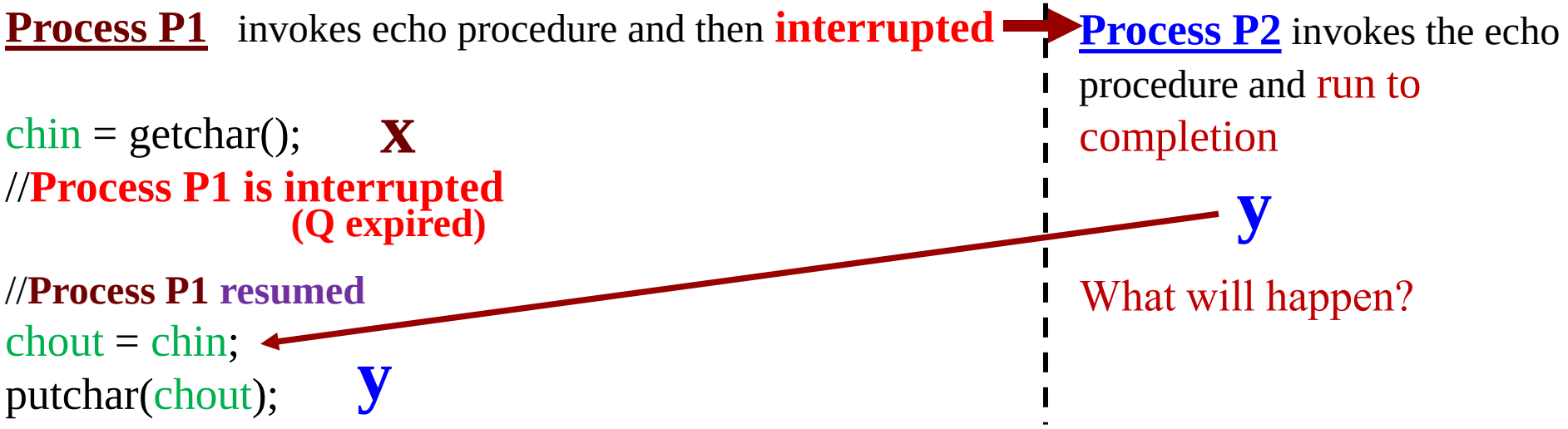
A Simple Example: Concurrent access to shared data

```
void echo()
{
//Access 2 shaered varaibles chin and chout
chin = getchar();
chout = chin;
putchar(chout);
}
```



chin and chout are
shared global variables

A Simple Example: Concurrent access to shared data



Single-processor multiprogramming system
Shared procedure

Such sharing can lead to problems. Consider the following sequence:

1. **Process P1** invokes the echo procedure and is interrupted immediately after getchar returns its value and stores it in chin. At this point, the most recently entered character, **x**, is stored in variable **chin**.
2. **Process P2** is activated and invokes the echo procedure, which runs to completion, inputting and then displaying a single character, **y**, on the screen.
3. **Process P1** is resumed, the value **x** has been overwritten in chin and therefore lost. Instead, **chin** contains **y**, which is transferred to **chout** and displayed.

- Thus, the first character is lost and the second character is displayed twice.
- The essence of this problem is the shared global variable, chin. Multiple processes have access to this variable.
- If one process updates the global variable and is then interrupted, another process may alter the variable before the first process can use its value.

Solution

Only one process at a time may be in that procedure

Concurrent/parallel access → serial access

Process P1

chin = getchar(); **x**

//Process P1 is interrupted (Q expired)

//**Process P1 resumed**

chout = chin;

putchar(chout); **y**

Process P2

.

Run to completion

y

Process P1

Lock chin (How???)

chin = getchar();

//Process P1 is interrupted (Q expired)

//**Process P1 resumed**

chout = chin;

putchar(chout); **x**

Unlock chin

Process P2

.

.

Lock chin // running → **P2 blocked (because p1 already locked chin)**

Solution

Only one process at a time may be in that procedure

1. Process P1 invokes the echo procedure and is interrupted immediately after the conclusion of the input function. At this point, the most recently entered character, **x**, is stored in variable chin.
2. Process **P2** is activated and invokes the echo procedure. However, because P1 is still inside the echo procedure, **P2** is blocked from entering the procedure.
3. At some later time, process P1 is resumed and completes execution of echo. The proper character, **x**, is displayed.
4. When P1 exits echo, this removes the block on **P2**. When **P2** is later resumed, the echo procedure is successfully invoked.

- This example shows that it is necessary to protect shared global variables (and other shared global resources) and the only way to do that is to control the code that accesses the variable.
- If we impose the discipline that only one process at a time may enter echo, and that once in echo the procedure must run to completion **before** it is available for another process, then the type of error just discussed will not occur.

Process P1

Lock chin (How???)
chin = getchar();
//Process P1 is interrupted (Q expired)

//**Process P1 resumed**
chout = chin;
putchar(chout);
Unlock chin

Process P2

.
.
Lock chin // running → **P2 blocked (because p1 already locked chin)**

- Access to **shared data** is an advantage and disadvantage **because** it allows **Concurrent access** to **shared data**.
- Solution: **Parallel** access→ **Serial** access How ?
  - Access to **shared data** requires both **Lock** and **Unlock**
  - When Access to **shared data** :
    1) Lock **data** first If succeeded, no others threads can access **shared data** (no update)
    2) Unlock **data** to allow others threads access shared data

- Is the code **dependent on timing** of **P1** and **P2** (**dependent** which one will get the processor first) **???**

No

Process P1

```
.
chin = getchar();
.
chout = chin;
putchar(chout);
.
```

Process P2

```
.
.
chin = getchar();
chout = chin;
.
putchar(chout);
```

Multiprocessor system, P1 and P2 each on a separate processor invoking the echo proc.

- In a multiprocessor system, the same problems of protected shared resources arise, and the same solution works.
- Processes P1 and **P2** are both executing, each on a separate processor.
- Both processes invoke the echo procedure.
- Events on the same line take place in parallel
- The result is that the character input to P1 is lost before being displayed, and the character input to **P2** is displayed by both P1 and **P2**.

Operating System Concerns

- Keep track of active processes
  - Done with the use of process control blocks
- Allocate and deallocate resources
  - Processor time
  - Memory
  - Files
  - I/O devices
- Protect data and resources of each process against unintended interference by other processes
- Result of process must be independent of the speed of execution of other concurrent processes.

To understand how the issue of **speed independence** can be addressed, we need to look at the ways in which processes can interact.

Process Interaction
Degree to which they are aware of each other's existence

1) Processes unaware of each other
   - These are independent processes that are not intended to work together
   - Competition among processes for resources
   - For example, two independent applications may both want to access the same disk or file or printer. The OS must regulate these accesses.

2) Processes indirectly aware of each other
   - Cooperation among processes **by sharing** (share access to the same object)

3) Process directly aware of each other
   - Cooperation among processes to work jointly on some activity by communication (communicate by process IDs)

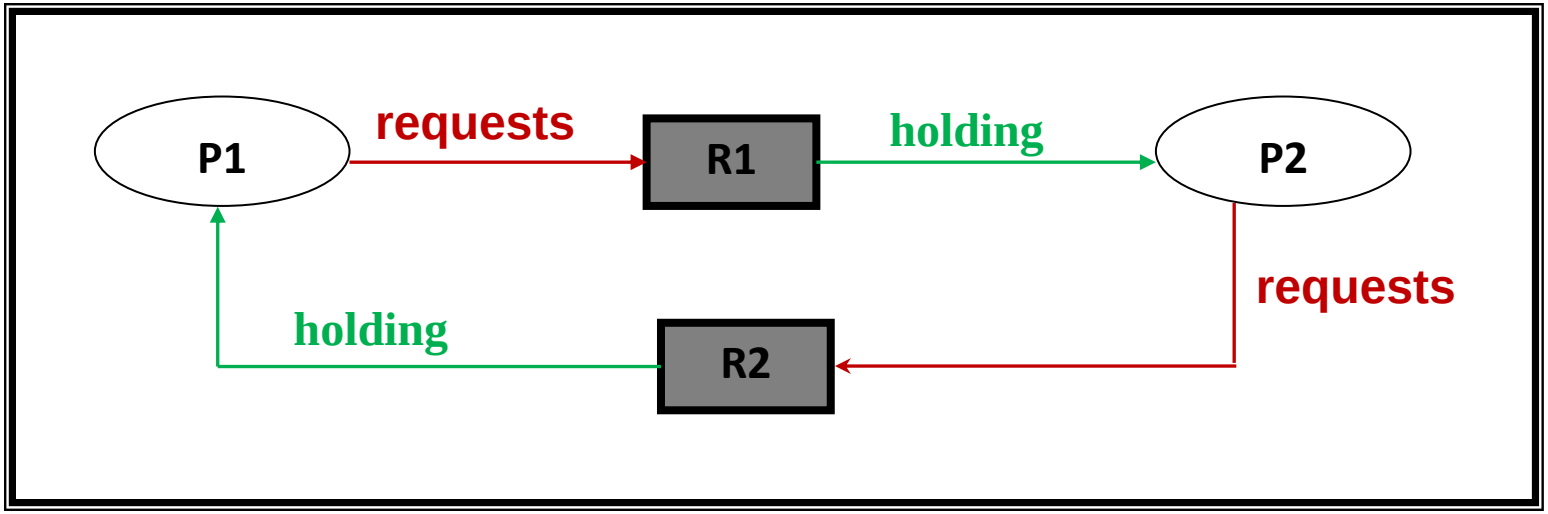
Competition Among Processes for Resources

Three control problems must be faced:

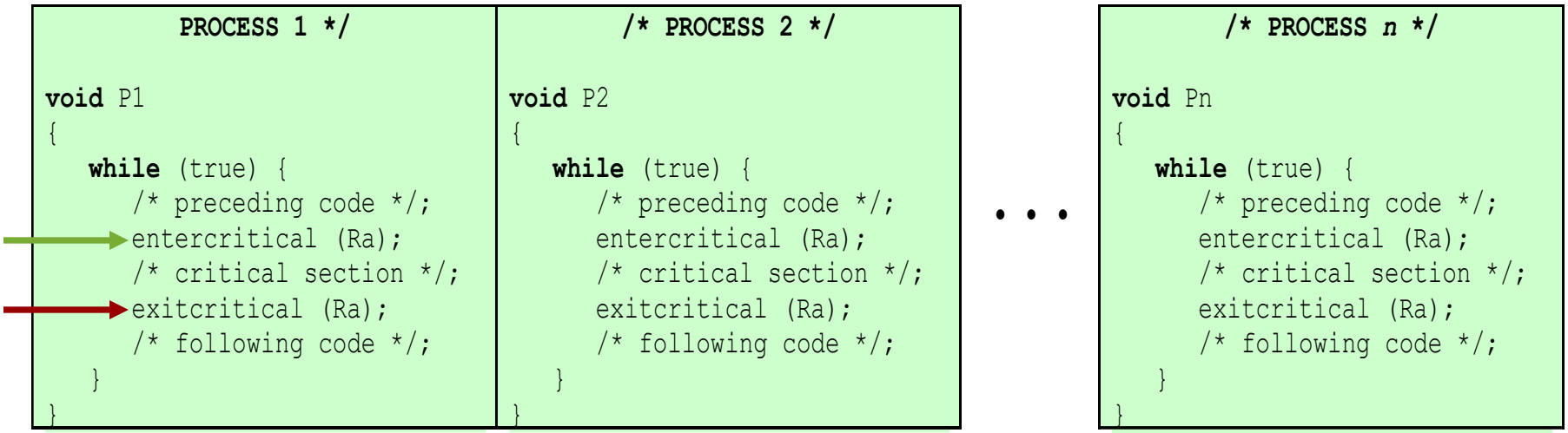
1) The need for Mutual Exclusion
   - Critical section
     - ➢ Suppose two or more processes require access to a single non-sharable resource, such as a printer (critical resource).
     - – **The portion of the program** that uses a critical resource is called critical section
     - – Only one program at a time is allowed in its critical section (No Concurrent Access)
     - – Example **only** one process at a time is allowed to send command to the printer (**controlled by** Print manager i.e. parallel → serial) **otherwise**, lines from competing processes will be interleaved.
2) Deadlock
3) Starvation
   - A process keeps waiting forever to access the critical section

Deadlock

- Consider two processes, P1 and P2, and two resources, R1 and R2.
- Each process needs access to both resources to perform its function.
- OS assigns R1 to P2, and R2 to P1.
- Each process is waiting for one of the two resources.
- Neither will release the resource that it already owns until it has acquired the other resource.
- The two processes are deadlocked.



Mutual Exclusion mechanism in abstract terms



```
/* PROCESS 1 */

void P1
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

```
/* PROCESS 2 */

void P2
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

. . .

```
/* PROCESS n */

void Pn
{
   while (true) {
      /* preceding code */;
      entercritical (Ra);
      /* critical section */;
      exitcritical (Ra);
      /* following code */;
   }
}
```

- There are n processes to be executed **concurrently**.
- Each process includes critical section that operates on some resource **Ra**
- Because all processes access the same resource **Ra**, it is desired that only one process at a time be in its critical section.
- To enforce mutual exclusion, **two functions** are provided: **entercritical** and **exitcritical**
- Each **function** takes as an argument the name of the resource that is the subject of competition.
- Any process that attempts to enter its critical section **while** another process is in its critical section, for the same resource, is made to **wait**.

Cooperation Among Processes by Sharing

- Access to shared data
- Data items accessed in reading and writing modes
- Writing must be mutually exclusive
- Critical sections are used to provide data integrity → data coherence

P1:

a = a+1;

b = b+1;

P2:

b = 2 * b;

a = 2 * a;

Maintain a=b

Any process that **updates** one value must also **update** the other to maintain the relationship.

Cooperation Among Processes by Sharing

Consider the following concurrent execution sequence:

```
a = a + 1;
b = 2 * b;
b = b + 1;
a = 2 * a;
```

- At the end of this execution sequence, the condition a = b no longer holds.
- For example, if we start with **a = b = 1**, at the end of this execution sequence we have a = 4 and b = 3.
- The problem can be avoided by declaring the entire sequence in each process to be a critical section.
- **The same abstract functions** of **entercritical** and **exitcritical** can be used here.
- In this case, the argument for the functions could be a variable.

Cooperation Among Processes by Communication

- Messages are passed
  - Because **nothing is shared** between processes in the act of passing messages, Mutual exclusion is not **a control requirement**
- Possible to have deadlock
  - Each process waiting for a message from the other process
- Possible to have starvation
  - Two processes (P1 and P2) repeatedly sending message to each other while another process (P3) **waits** for a message from P1.
  - There is **no** deadlock, because P1 remains active, but P3 is starved.

Requirements for Mutual Exclusion

1) Only one process at a time is allowed in the critical section for a resource
2) A process that halts in its non-critical section must do so **without** interfering with other processes.
3) No deadlock or starvation
4) A process must not be delayed access to a critical section **when** there is no other process using it.
5) No assumptions are made about **relative process speeds** or number of processes
6) A process remains inside its critical section for a finite time only

How can the requirements be satisfied?

1) Software approaches
   - Processes (Programmer) responsibility
   - No support from OS or programming language
2) Hardware support
   - Special-purpose machine instructions (in instruction set)
3) Support of OS and programming language
   - OS provides Data structure (Kernel Level)
   - The programmer can access it **through** programming language (socket is a Data structure (Kernel OS) accessed through Java API)
   - e.g. Semaphore

Mutual Exclusion: Software Approaches

```
// A is Local
int A= 5;
if (A < 10)
then …
else …

// A is shared variable
if (A < 10)
{
   (Q expired) Runing→Ready
    A can be changed by other
    process/threads
          ...
}
else …
```

if → Repetition

If (condition) {...} → loop and negate condition and end with ;

```
while (! Condition);   //loop without body
while (A >= 10); //loop until A < 10 (Busy Waiting)
// critical section
access to shared variable (No Concurrent Access)
```

We need to control access to A such that A still <10 during execution if statement (how ???)

Mutual Exclusion: Software Approaches

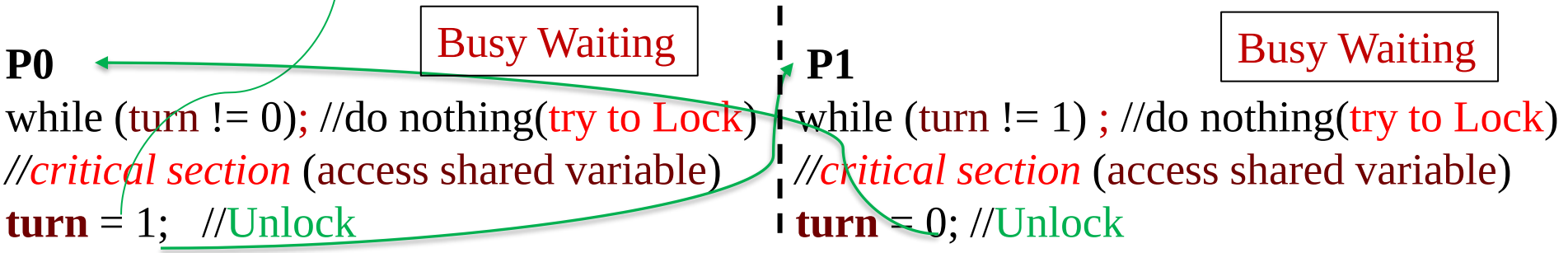
- Assume elementary mutual exclusion at memory access level
  - Simultaneous access to same location in memory are **serialized** by some sort of memory arbiter.
- Dekker's Algorithm (reported by Dijkstra)
  - 4 attempts to reach the correct solution!
- Peterson's Algorithm

Dekker's Algorithm: First Attempt

- Busy Waiting
  - Process is always checking to see if it can enter the critical section
  - Process can do nothing productive **until** it gets permission to enter its critical section.

Dekker's Algorithm: First Attempt

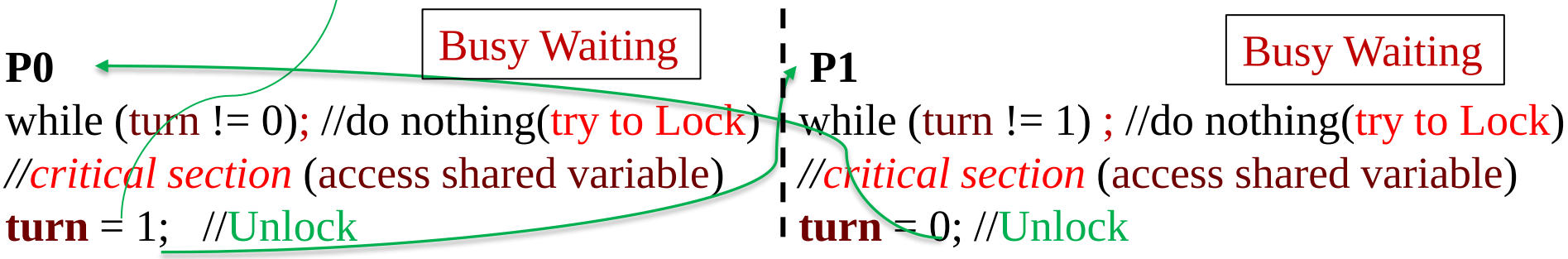
P0 and P1 access shared variable
Control variable turn
turn=0 → access to P0
turn=1 → access to P1



- Shared global variable *turn* indicates who is allowed to enter next
- Process P0 or P1wishing to execute its critical section first examines value of turn.
- If the value of turn is equal to the process NO, then the process may proceed to its critical section.
- **Otherwise**, it is forced to wait and repeatedly reads the value of turn until it is allowed to enter its critical section.
- **After** a process has gained access to its critical section, and after it has completed that section, it must update the value of turn to that of the other process.

Dekker's Algorithm: First Attempt

P0 and P1 access shared variable
Control variable turn
turn=0 → access to P0
turn=1 → access to P1



Is P0 and P1 in critical section at the same time ?

int turn = 0;

P0 → Processor

P0 only enter critical section
While **P0** in critical section
P1can not enter critical section Why?

P1 → Processor

P1 will not able to enter critical section
but can enter critical section **when** **P0** is allocated Processor and changes **turn** to 1

int turn = 1;

P0 → Processor

P1 → Processor

Is Mutual Exclusion is achieved? justify your answer

yes, No two processes in critical section

Is deadlock occurred? justify your answer

No, Turn is a control

Is this code dependent on which one will get processor first? justify your answer

No, Turn will control which one will first enter critical section.

If we have 3 processes Pls, write the code

Advantages

Mutual Exclusion is achieved

Disadvantages

1) Not the optimum solution (based on Busy Waiting)
2) The order is determined by **programmer**
3) If one process fails, the other process is permanently blocked. This is true whether a process fails in its critical section or outside of it.

Coroutine

- The foregoing construction is that of a coroutine.
- A coroutine is a concurrency design pattern that you can use to simplify code that executes asynchronously (allow execution to be suspended and resumed).
- Coroutines are designed to be able to pass execution control back and forth between themselves.
- While this is a useful structuring technique for a single process, it is inadequate to support concurrent processing.

Second Attempt

- Each process has a flag to indicate it is in the critical section
- Each process can examine the other's status but cannot alter it
- When a process wants to enter the critical section is checks first the other processes flags until that flag has the value false, indicating that the other process is not in its critical section.
- If no other process is in the critical section, it sets its own flag to true and proceeds to its critical section.
- When it leaves its critical section, it sets its flag to false.

Second Attempt

- "Busy" Flag → state information

P0	P1
while (flag[1]) ; //do nothing	while (flag[0]); //do nothing
flag[0] = true;	flag[1] = true;
//critical section	*//critical section*
flag[0] = false;	flag[1] = false;

- Each process has a *flag* to indicate it is in the critical section
- Fails mutual exclusion if processes are in *lockstep*
- What happens if a process fails inside its critical section or after setting its flag to true just before entering its critical section? the other process is permanently blocked.
- What happens if one process fails outside the critical section, including the flag-setting code? the other process can enter its critical section as often as it likes, because the flag of the other process is always false.
- This method does not guarantee mutual exclusion
  - Each process can check the flags and then proceed to enter the critical section at the same time.

Third Attempt

- Set flag to enter critical section before check other processes
- If another process is in the critical section when the flag is set, the process is blocked until the other process releases the critical section.
- Deadlock is possible when two process set their flags to enter the critical section.
  - Now each process must wait for the other process to release the critical section.

Third Attempt

- Busy Flag Modified

P0

```
flag[0] = true;
while ( flag[1] ) ; //do nothing
//critical section
flag[0] = false;
```

P1

```
flag[1] = true;
while ( flag[0] ); //do nothing
//critical section
flag[1] = false;
```

- Guarantees mutual exclusion
- Deadlock if processes are in lockstep (both processes set their flags to true before either has executed the while statement)

- ➢ As before, if one process fails inside its critical section, including the flag-setting code controlling the critical section, then the other process is blocked.
- ➢ If a process fails outside its critical section, then the other process is not blocked.

Fourth Attempt

- Each process sets its flag to indicate its desire to enter its critical section **but** is prepared to reset the flag to defer to the other process.
- Other processes are checked. If they are in the critical region, the flag is reset and later set to indicate desire to enter the critical region.
- This is repeated until the process can enter the critical region.
- Mutual exclusion is still guaranteed, using similar reasoning to that followed in the discussion of the third attempt.

Fourth Attempt

- It is possible for each process to set their flag, check other processes, and reset their flags.

P0 sets flag[0] to true.
Pl sets flag[1] to true.
P0 checks flag[1].
Pl checks flag[0].
P0 sets flag[0] to false.
Pl sets flag[1] to false.
P0 sets flag[0] to true.
Pl sets flag[1] to true.

- This scenario will not last very long so it is not deadlock. It is undesirable.

- Because any alteration in the relative speed of the two processes will break this cycle and allow one to enter the critical section. This condition is referred to as **livelock**.
- Recall that deadlock occurs when a set of processes wishes to enter their critical sections, but no process can succeed.
- With livelock, there are possible sequences of executions that succeed, but it is also possible to describe one or more execution sequences in which no process ever enters its critical section.

Fourth Attempt

•Busy Flag Again

P0

```
flag[0] = true;
while ( flag[1] )
{
        flag[0] = false;
        /*delay*/
        flag[0] = true;
}
/*critical section*/
flag[0] = false;
```

P1

```
flag[1] = true;
while ( flag[0] )
{
  flag[1] = false;
/*delay*/
flag[1] = true;
}
/*critical section*/
flag[1] = false;
```

Correct Solution

- Each process gets a turn at the critical section
- If a process wants the critical section, it sets its flag and may have to wait for its turn.

Correct Solution

```
 boolean flag [2];
int turn;

void main ()
{
  flag [0] = false;
  flag [1] = false;
  turn = 1;
  parbegin (P0, P1);
}
```

```
void P0() {
  while (true)
  {
    flag [0] = true;
     while (flag [1]) {
       if (turn == 1)
       {
          flag [0] = false;
          while (turn == 1) ;
            // do nothing
            flag [0] = true;
       }
                              }
/* critical section */;
turn = 1;
flag [0] = false;
/* remainder */;
  }
                    }
```

```
void P1() {
  while (true)
  {
    flag [1] = true;
     while (flag [0]) {
       if (turn == 0)
       {
          flag [1] = false;
          while (turn == 0) ;
            // do nothing
            flag [1] = true;
       }
                              }
/* critical section */;
turn = 0;
flag [1] = false;
/* remainder */;
  }
                    }
```

- PARBEGIN statement is a higher-level language construct for specifying concurrency. All statements can be executed concurrently.
- Enables several activities or processes to run simultaneously. It separates the code into sections that can run concurrently and independently.

The construct parbegin (Pl, P2, . . . , Pn) means the following: suspend the execution of the main program; initiate concurrent execution of procedures Pl, P2, . . . , Pn; when all of Pl, P2, . . . , Pn have terminated, resume the main program.

Peterson's Algorithm

As before, the global array variable flag indicates the position of each process with respect to mutual exclusion, and the global variable turn resolves simultaneity conflicts.

Peterson's Algorithm

P0

```
while (true)
{
flag[0] = true;
turn = 1;
while ( flag[1] &&turn == 1 );
 // do nothing
/*critical section*/
flag[0] = false;
}
```

P1

```
while (true)
{
   flag[1] = true;
    turn = 0;
while ( flag[0] && turn == 0 );
// do nothing
/*critical section*/
flag[1] = false;
}
```

```
void main() {
flag [0] = false;
flag [1] = false;
parbegin (P0, P1); }
```

Can you show that mutual exclusion is preserved?

- Consider process P0.

- Once it has set flag[0] to true, Pl cannot enter its critical section.
- If Pl already is in its critical section, then flag[1] = true and P0 is blocked from entering its critical section.
- Suppose that P0 is blocked in its while loop.
- This means that flag[1] is true and turn = l. P0 can enter its critical section when either flag[1] becomes false or turn becomes 0.

Mutual Exclusion: Hardware Support

1) Interrupt Disabling

- ✓ A process runs **until** it invokes an OS service or until it is **interrupted**
  - Disabling interrupts **guarantees** mutual exclusion
  - This capability can be provided in the form of primitives defined by the OS kernel for disabling and enabling interrupts.
  - A process can then enforce mutual exclusion in the following way:

```
while (true) {
        /* disable interrupts */;
        /* critical section */;
        /* enable interrupts */;
        /* remainder */;
{
```

Mutual Exclusion: Hardware Support

Disadvantages

1) The efficiency of execution could be **degraded**

   –**Processor** is limited in its ability to **interleave** processes

2) Will not work in a multiprocessor architecture.

   – Disabling interrupts on one processor will not guarantee mutual exclusion.

2) Special Machine Instructions

- ✓ At the hardware level access to a memory location excludes any other access to that same location.
- ➢ Processor designers have proposed **several machine instructions** that carry out two actions atomically.
  - The instruction is treated as a single step that cannot be interrupted
  - Performed in a single instruction cycle (Not subject to interference from **other instructions**)

  Such as:

  – Reading and writing of a single memory location

  – Reading and testing of a single memory location
- ➢ During execution of **the instruction**, access to the memory location is blocked for **any other instruction** referencing that location.

Test and Set Instruction

```
//i shared variable
boolean testset (int i) {
        if (i == 0)  // not Locked{
        i = 1;   // Lock
        return true;
                    }
        else {
        return false;
            }
```

Test and Set Instruction

```
//i shared variable
boolean testset (int i) {
        if (i == 0)  // not Locked{
        //Q expired
        i = 1;  // Lock
        return true;
                    }
        else {
        return false;
            }
```

Test and Set is instruction in instruction set

Instruction is atomic (all or non at all)

Carried out atomically, not subject to interruption

```
/* program mutualexclusion */
const int n = /* number of processes */;
int ctrlvar;
void P(int x) {
while (true)
{
  while (!testset (ctrlvar)) ; // do nothing (can not set)
     // critical section      // mutual exclusion
          ctrlvar = 0; // unLock
     /* remainder */
}
                    }
void main()
{
ctrlvar = 0;
parbegin (P(1), P(2), . . . ,P(n)); }
```

Test and Set Instruction

```
//i shared variable
boolean testset (int i) {
if (i == 0)  // not Locked{
i = 1; // Lock
return true;                }
else {
return false;                 }
```

- The first process/thread selected by processor will enter **critical section.**
- A process that finds **ctrlvar** equal to zero can enter its **critical section.**

The construct **parbegin** (Pl, P2, . . . , Pn) means the following: suspend the execution of the main program; initiate concurrent execution of procedures Pl, P2, . . . , Pn; when all of Pl, P2, . . . , Pn have terminated, resume the main program.

Advantages

1) Applicable to any number of processes on either a single processor or multiple processors sharing main memory
2) It is simple and therefore easy to verify
3) It can be used to support multiple critical sections each can be defined by its own variable.

Disadvantages

1) Busy-waiting while a process is waiting for access to a critical section, consumes processor time.
2) Starvation is possible when a process leaves a critical section and more than one process is waiting, the selection of a waiting process is arbitrary. Thus, some process could indefinitely be denied access.

Disadvantages

3) Deadlock

- If a low priority process has the critical region and a higher priority process needs, the higher priority process will obtain the processor to wait for the critical region.

- Consider the following scenario on a single-processor system.
  - Process P1 executes the special instruction and enters its critical section.
  - P1 is then interrupted to give the processor to P2, which has higher priority.
  - If P2 now attempts to use the same resource as P1, it will be denied access because of the mutual exclusion mechanism. Thus it will go into a busy waiting loop.
  - However, P1 will never be dispatched because it is of lower priority than another ready process, P2.

Exchange Instruction

```
void exchange(int *register, int *memory) {
int temp;
temp = *memory;
*memory = *register;
*register = temp;
}
```

```
/* program mutualexclusion */
int const n = /* number of processes */;
int bolt;
void P(int i)
{
while (true)
int keyi = 1;
Do
exchange (&keyi, &bolt)
while (keyi != 0);
/* critical section */;
bolt = 0;
/* remainder */;
}
}
void main() {
bolt = 0;
parbegin (P(1), P(2), ..., P(n));
}
```

- Each process uses a local variable **key** that is initialized to 1.
- The only process that may enter its critical section is one that finds bolt equal to 0.
- It excludes all other processes from the critical section by *setting bolt to 1*.
- When a process leaves its critical section, it resets bolt to 0, allowing another process to gain access to its critical section.
- If bolt = 0, then no process is in its critical section.
- If bolt = 1, then exactly one process is in its critical section, namely the process whose **key** value equals 0.

Mutual Exclusion: OS and programming support

- Semaphores
- Monitors
- Message Passing

Semaphores

Control variable that control access of shared variable

- Data structure implemented at kernel level and accessed by API in programming languages to achieve mutual exclusion.
- Two or more processes can cooperate by means of simple **signals**
- A process is forced to stop at a specified place **until** it has received **a specific signal**
- Special variable called a semaphore is used for **signaling**
  - To receive **a signal via** semaphore *s*, a process executes primitive ***wait****(s)* *(P → (proberen) in dutch)* ***test*** ***(decrements semaphore value)***
  - To transmit **a signal via** a semaphore *s*, a process executes primitive ***signal****(s)* *(V → (verhogen) in dutch)* ***(increment semaphore value)***

Wait and **signal** are **instructions** at the level of instruction set in the processor design (cannot be interrupted (Atomic .i.e. all or non at all)

Semaphores

- Semaphore is a variable that has **an integer value**
  - May be initialized to a nonnegative number
  - ***wait*** operation **decrements semaphore value**
    - If value becomes **negative**, process executing the *wait* is blocked **Otherwise**, the process **continues execution**.
  - ***signal*** operation **increments semaphore value**
    - If value is not positive, a process blocked by a *wait* operation is unblocked.

- *If a process is waiting for a signal, it is suspended until that signal is sent.*
- Queue is used to hold processes waiting on the semaphore

Counting semaphores

Control access to a given recourse of finite number of instances

- **Semaphore counter is initialized to the # of available resources**
- **Each process wishes to use a resource perform a wait () operation on semaphore (Decrementing the count)**
- **When count goes to 0, all resources are used**
- **After that, processes that wish to use a resource will block until the count becomes >0**

Counting semaphores

struct semaphore {
int count;
queueType queue; };
Semaphore s, s.count = 5 //Assume we have 5 network printer
P1 requests to print wait(s) Decrement 5 → 4
P2 requests to print wait(s) Decrement 4 → 3
P3 requests to print wait(s) Decrement 3 → 2
P4 requests to print wait(s) Decrement 2 → 1
P5 requests to print wait(s) Decrement **1 → 0** //no more printers
P6 requests to print wait(s) Decrement 0 → -1 //Running → Blocked → add this process in s.queue
P7 requests to print wait(s) Decrement -1 → -2 //Running → Blocked → add this process in s.queue

The absolute value is the number of awaiting processes

//P1 finished signal(S) increment -2 → -1 remove a process from s.queue and move it to Ready state
//P2 finished signal(S) increment **-1 → 0** remove a process from s.queue and move it to Ready state

Counting semaphores

```
struct semaphore {
int count;
queueType queue;
};
int n  // number of units of shared resource;
Semaphore s, s.count = n;
void Wait(semaphore s) {
s.count--;
if (s.count < 0) {   Q expired
// place this process in s.queue
// block this process  }
}
void Signal(semaphore s) {   Q expired
s.count++;
if (s.count <= 0)   // -1 → 0 {
// remove a process P from s.queue
// place process P on ready list  }
}
```

When the process that initially entered its critical section departs, **s** is incremented and one of the blocked processes (if any) is removed from the queue of blocked processes associated with the semaphore and put in a Ready state.

Wait and **signal** are **instructions** at the level of instruction set in the processor design (cannot be interrupted (Atomic .i.e. all or non at all)

Mutual Exclusion using Semaphores

```
/* program mutualexclusion */
const int n = /* number of processes */;
Semaphore s, s.count = 1;
void P(int i) {
while (true) {
Wait(s);  //lock 1 → 0
// critical section     // can be a shared file/buffer/resource
Signal(s);  // unlock
// remainder
}
}
void main()  {
parbegin (P(1), P(2), . . . , P(n)); }
```

One process is allowed in its critical section at a time

Figure 5.9 Mutual Exclusion Using Semaphores

➢ To overcome the need for busy waiting, we use *the wait() and signal()* as follows :

- When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, *the process can block itself*.
- The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Then control is transferred to the CPU scheduler, which selects another process to execute.
- A process that is blocked, waiting on a semaphore S, should be restarted when some other process executes a signal operation.
- The process is restarted by a wakeup operation, which changes the process from the waiting state to the ready state and placed in the ready queue.
- (The CPU may or may not be switched from the running process to the newly ready process, depending on the CPU-scheduling algorithm.)

- Note: we have not completely eliminated busy waiting with this definition of the wait() and signal() operations.
- Rather, we have moved busily waiting from the entry section to the critical sections of application programs.
- Furthermore, we have limited busy waiting to the critical sections of the wait() and signal() operations, and these sections are short (if properly coded, they should be no more than about ten instructions).
- Thus, the critical section is almost never occupied, and busy waiting occurs rarely, and then for only a short time.
- An entirely different situation exists with application programs whose critical sections may be long (minutes or even hours) or may almost always be occupied.
- In such cases, busy waiting is extremely inefficient.

Disadvantages

Deadlocks and Starvation

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for **an event** that can be caused only by one of the waiting processes.
- The event in question is the execution of a signal operation. When such a state is reached, these processes are said to be deadlocked.

Consider two processes, **Po** and **PI**, each accessing **two semaphores**, **S** and **Q**, set to the value **1**:

P0	**P1**
wait (S)	wait (Q)
wait (Q)	wait (S)
.	
.	
.	
signal (S)	signal(Q)
signal (Q)	signal (S)

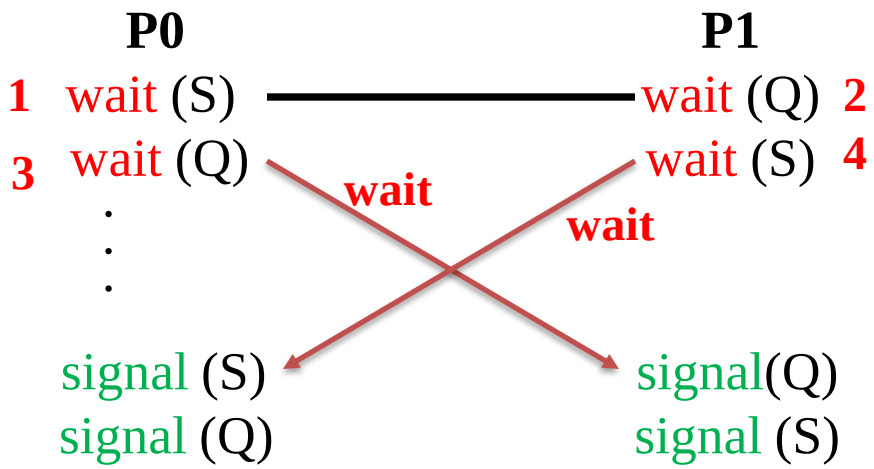
signal operations can not be executed → P0 and P1 are in deadlock

- Suppose that Po executes wait (S) and then P1 executes wait (Q).
- When Po executes wait (Q), it must wait until P1 executes signal (Q).
- Similarly, when P1 executes wait (S), it must wait until Po executes signal(S).
- Since these signal() operations cannot be executed, Po and P1 are deadlocked.

 Another problem related to deadlocks is indefinite blocking or starvation:

A situation in which processes wait indefinitely within the semaphore (A process may never be removed from the semaphore queue in which it is suspended)

➢ Indefinite blocking may occur if we remove processes from the list associated with a semaphore in LIFO (last-in, first-out) order.

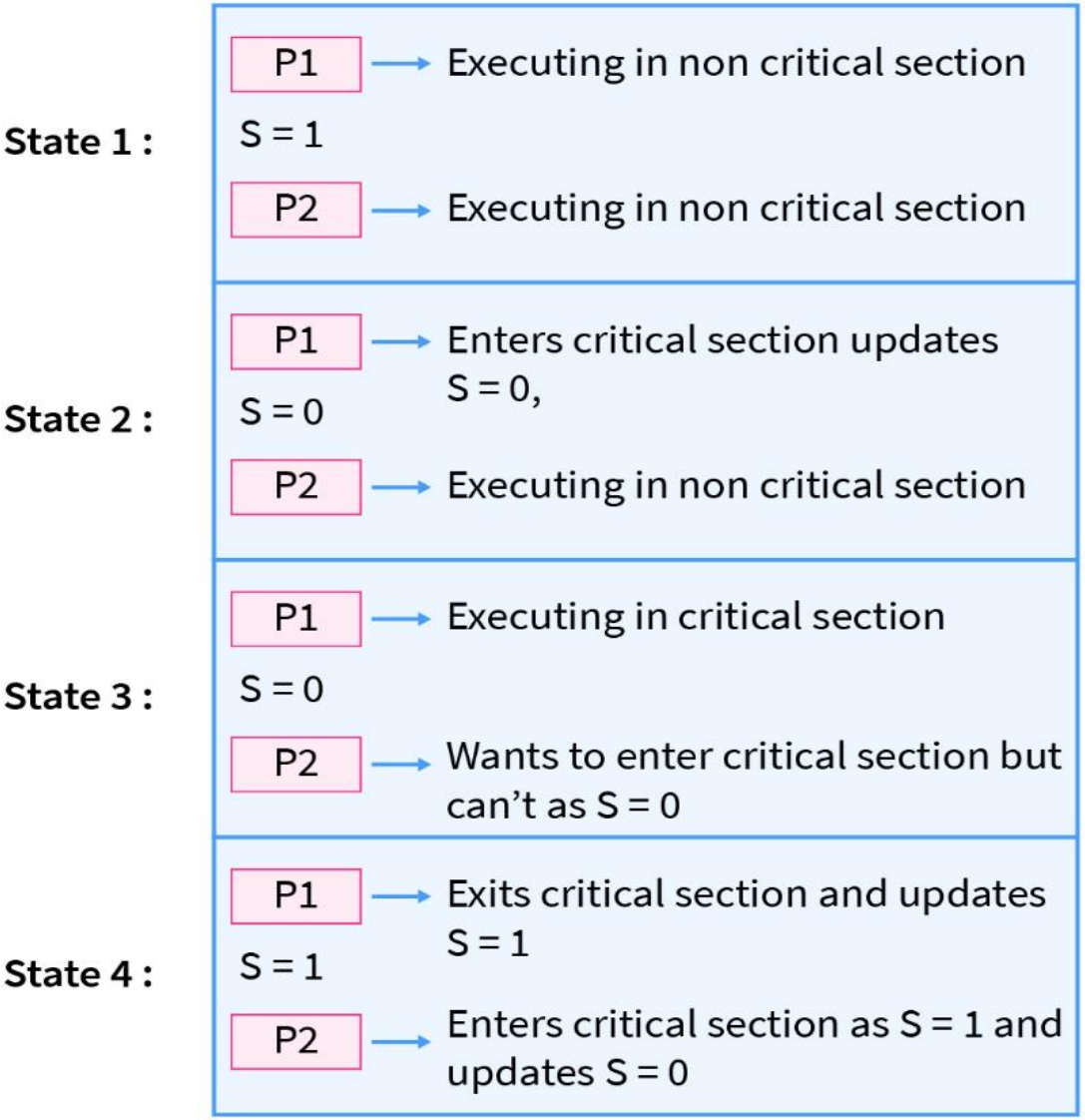


Binary semaphore: state on and off

Control access to a given recourse with only one instance

1) A binary semaphore is initialized to **1** and only takes the values **1** and **0** during the execution of a program (lock/unlock – available /unavailable)
2) The WaitB operation checks the semaphore value.
   - If the value is **1**, process can proceed to the critical section and the value is changed to **0**
   - If the value is **0**, process cannot proceed to the critical section and blocked.
3) The SignalB operation checks to see if any processes are blocked on this semaphore (semaphore value equals **0**).
   - If so, then a process blocked by a WaitB operation is unblocked.
   - If no processes are blocked, then the value of the semaphore is set to **1**.

This is how mutual exclusion is achieved using binary semaphore i.e. **both processes cannot access the critical section at the same time.**



Binary semaphore primitives

```
struct binary_semaphore {
enum {zero, one} value;
queueType queue;
};
void WaitB(binary_semaphore s)
{
if (s.value == one)
s.value = zero;
else {
/* place this process in s.queue */;
/* block this process */;
}
}
void SignalB(semaphore s)
{
if (s.queue is empty())
s.value = one;
else {
/* remove a process P from s.queue */;
/* place process P on ready list */;
}
}
```

Strong and Weak Semaphore

- For both counting semaphores and binary semaphores, a queue is used to hold processes waiting on the semaphore.
- The question arises of the **order** in which processes are removed from such queue.
- The fairest removal policy is first-in-first-out (**FIFO**): The process that has been blocked the longest is released from the queue first; a semaphore whose definition includes this policy is called **a strong semaphore**.
- A semaphore that does not specify the **order** in which processes are removed from the queue is **a weak semaphore**.

- **Strong semaphore** versus **weak semaphore** (specification of **order** of processes to be removed from queue)
- **Strong semaphores** guarantee freedom from starvation, while weak semaphores do not.

Using Semaphore for synchronization
Control the order of execution (No Sharing)

Assume that we want P2 first then p1

Process P1

```
Wait(s);
printf(" P1");
```

Process P2

```
printf(" P2");
Signal(s);
```

Q1: Control point? 1 **Semaphore S**

Q2:Initial value s.count? 0 **(0 → -1)** //Running → Blocked

Q3: how to use wait/signal

Suppose P1 executed before P2

P1: wait **s.count** 0 → -1 **P1 Blocked**

When P2 later executed

P2:signal **s.count** -1 → 0

P1 will be form Blocked to Ready

Suppose P2 executed before P1

P2 : signal **s.count** 0 → 1

When P1 later executed

P1 : wait **s.count** 1 → 0

Assume that we want P3 first then p1 then P2

Process P1

```
Wait(s1);
printf(" P1");
Signal(s2);
```

Process P2

```
Wait(s2);
printf(" P2");
```

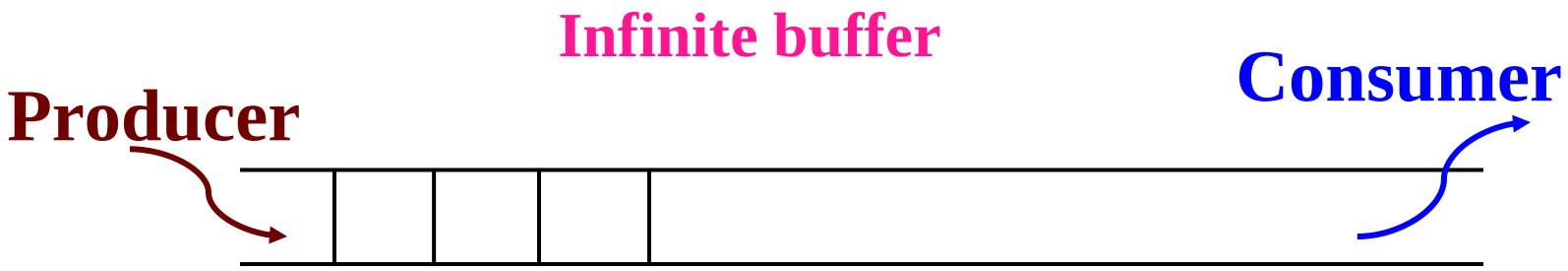
Process P3

```
printf(" P3");
Signal(s1);
```

Q1: Control point? 2 **Semaphore S1, S2**
Q2:Initial value s.count? **s1.count = s2.count = 0**
Q3: how to use wait/signal

Producer/Consumer Problem

- One or more **producers** are generating data and placing in **a buffer**
- A single **consumer** is taking items out of the **buffer** one at time
- Only one **producer** or **consumer** may access the **buffer** at any one time



Producer/Consumer Problem

a) Producer and consumer can not access buffer at the same time (Mutual Exclusion)
b) Consumer can not read empty buffer (synchronization)

a) Mutual Exclusion

Semaphore s , s.count = 1 // wait(s) lock 1 → 0

b) Synchronization

Semaphore n (number of items in the buffer),n.count= 0 (no item)//wait(n) 0 → -1 blocked

Producer

```
while (true) {
produce()
wait(s) //lock buffer
//critical section
  Store()
signal(n)
signal(s) //unlock buffer
}
```

```
signal(s) //unlock buffer
signal(n)
```

Q expired

Consumer

```
while (true) {
wait(n) / test if an item to be read (if no → Block)
wait(s) //lock buffer
//critical section
read()
consume()
signal(s) //unlock buffer
}
```

```
wait(s)
wait(n)
```

Consumer: wait(s) //lock buffer 1 → 0
Consumer: wait(n) // 0 → -1 **Block**
Producer: wait(s) // 0 → -1 **Block**
DEDLOCK

Is Mutual Exclusion and Synchronization are achieved? justify your answer.
Is there any probability that the producer and consumer at the critical section at the same time?

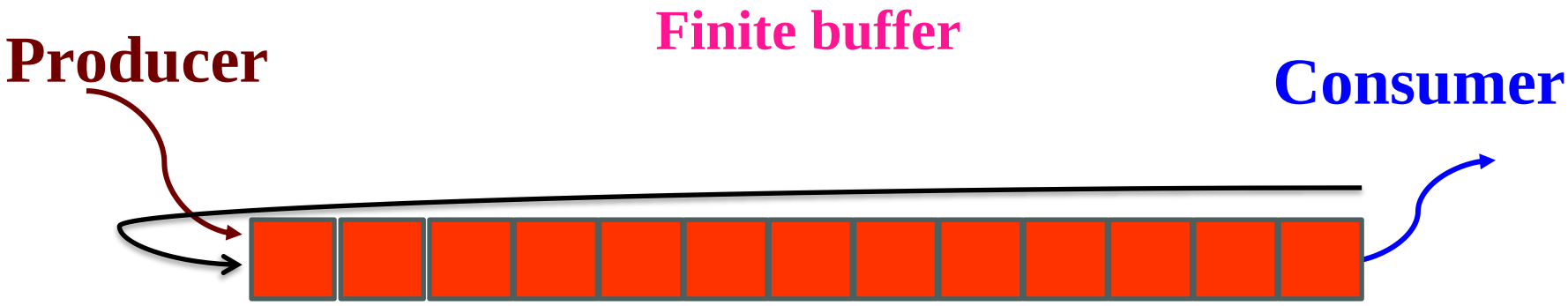
No, because the control by s both call wait (s)
Both can not lock buffer at the same time.

Also, not dependent on which one will get processor first .

Because, who get processor first will first lock buffer and controlled by n.

So, Mutual Exclusion and Synchronization are achieved

Finite buffer: Circular buffer



Finite buffer: Circular buffer

a) **Producer and consumer can not access buffer at the same time (Mutual Exclusion)**
   Semaphore s , s.count = 1 wait(s) lock 1 → 0

b) **Consumer can not read empty buffer (synchronization)**
Semaphore n (number of items in the buffer), n.count= 0 (no item)//wait(n) 0 → -1 blocked

c) **Producer can not overwrite locations not yet read by Consumer (synchronization)**
   Semaphore e (number of empty spaces in buffer) , e.count = buffer size

Producer

```
while (true) {
produce()
wait(e) //test if there is free location
wait(s) //lock buffer
//critical section
Store()
signal(n)
signal(s) //unlock buffer
}
```

Consumer

```
while (true) {
wait(n)  // test if an item to be read
wait(s) //lock buffer
//critical section
read()
consume()
signal(e)
signal(s) //unlock buffer
}
```

```
wait(s)
wait(e)
```



Producer : wait(s) // if buffer available it will lock **buffer**
Consumer: not yet read (no free locations)
Producer: wait(e) // **Block**
Consumer: wait(s) **Block**
DEDLOCK

2 Producers and one consumer (Infinite buffer)

Producer 1

wait(s) //lock buffer

signal(n)
signal(s) //unlock buffer

Producer 2

wait(s) //lock buffer

signal(n)
signal(s) //unlock buffer

Consumer

wait(n)
wait(s) //lock buffer
signal(s) //unlock buffer

Infinite buffer



No matter how many producers there are, no problem as long as ME is ensured

One Producer and 2 consumers (Infinite buffer)

Producer

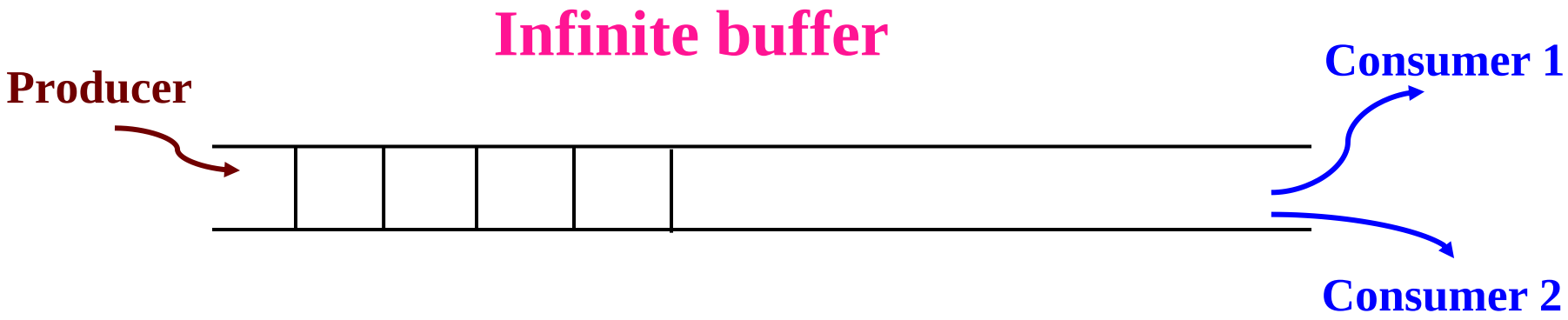
```
wait(s) //lock buffer
signal(n1)
signal(n2)
signal(s) //unlock buffer
```

Consumer 1

```
wait(n1)
wait(s) //lock buffer
signal(s) //unlock buffer
```

Consumer 2

```
wait(n2)
wait(s) //lock buffer
signal(s) //unlock buffer
```



Array of n semaphore for each consumer

In case of finite buffer, we need Semaphore e for each Consumer

Producer/Consumer using Binary Semaphores

Producer

```
do forever
produce item
waitB(s)
append
n++
if (n == 1)
    signalB(delay)
signalB(s)
```

Consumer

```
wait(delay)
do forever
waitB(s)
remove from queue
n--
m = n
signalB(s)
if (m = 0) waitB(delay)
```

n: number of items in the buffer, init to 0
s: semaphore used to enforce mutual exclusion, init to 1
delay: semaphore used to force **consumer** to wait if buffer is empty, init to 0

Producer/ consumer with Infinite Buffer

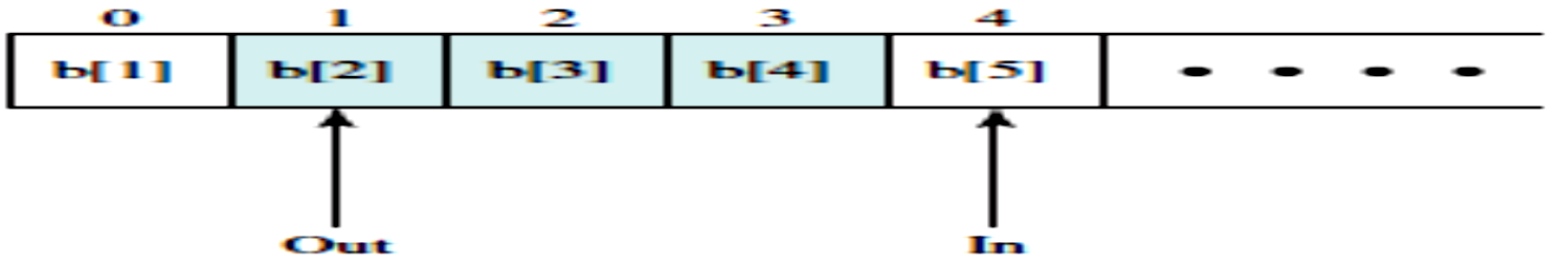
producer:

```
while (true) {
// produce item v /
b[in] = v;
in++;
}
```

consumer:

```
while (true) {
while (in <= out)
//do nothing /;
w = b[out];
out++;
// consume item w
}
```

Must not attempt to read from an empty buffer (in > out)



Note: Shaded area indicates portion of buffer that is occupied.

Figure 5.11 Infinite Buffer for the Producer/ Consumer Problem

Producer/ consumer with Circular Buffer

producer:

```
while (true) {
// produce item v
while ((in + 1) % n == out)
// do nothing;
b[in] = v;
in = (in + 1) % n
```

consumer:

```
while (true) {
while (in == out)
// do nothing;
w = b[out];
out = (out + 1) % n;
// consume item w
}
```

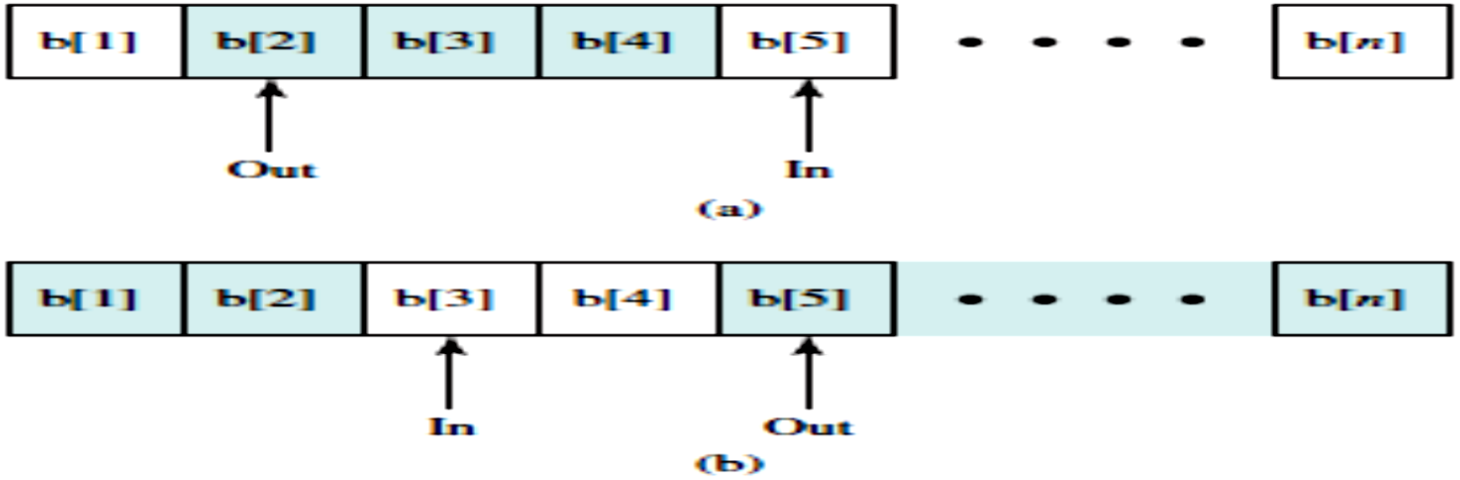


Figure 5.15 Finite Circular Buffer for the Producer/Consumer Problem


Any Questions ?
Thank you